MANIPAL UNIVERSITY JAIPUR

MANIPAL UNIVERSITY JAIPUR

Department of Information Technology

Software Engineering Laboratory

PROJECT REPORT

Government Schemes Matcher and Eligibility Tester

A PHP and MySQL web application for preliminary scheme matching

Submitted by

Divayom Sengar
Registration No. withheld
Section 5E
B.Tech Information Technology

20 October 2026

Declaration

I declare that this report describes the Government Schemes Matcher and Eligibility Tester project prepared for the Software Engineering Laboratory. The application and its demonstration records are presented as educational work. Any scheme examples supplied with the project are fictional and are not official government guidance.

Student: Divayom Sengar

Registration number withheld

Date: 20 October 2026

Abstract

This project is a database-driven web application for browsing scheme information and checking it against a user profile. It uses PHP, PDO, and MySQL or MariaDB. Users can search the catalogue, maintain optional profile details, and request an assessment that explains each configured eligibility condition. Administrators can maintain scheme records, categories, rules, documents, sources, and review status.

The eligibility engine reads typed rules from the database and reports PASS, FAIL, UNKNOWN, NOT APPLICABLE, or INVALID for each condition. It does not make an official decision. The included seed records are fictional and clearly marked as demonstration data. The project was checked with PHP syntax validation, 18 eligibility unit cases, and a local browser workflow using an isolated demonstration database.

Contents

1. Introduction

2. Requirements and scope

3. System design

4. Implementation

5. Website pages and demonstration

6. Testing and results

7. Installation and limitations

Conclusion

References

1. Introduction

1.1 Problem statement

Scheme details are published across different sources, and eligibility can depend on several facts about a person or household. A user may need to check age, income, state, occupation, education, or residence before deciding whether to apply. Comparing these conditions one scheme at a time can be difficult, especially when some information is missing.

1.2 Project aim

The aim is to build a local web application that keeps scheme information in a database and gives users a clear, preliminary comparison between their saved profile and the rules configured for a scheme.

1.3 Objectives

- Provide a searchable scheme catalogue with categories, benefit descriptions, documents, sources, and review status.
- Allow users to keep optional profile information and run assessments against saved rules.
- Explain each condition as pass, fail, unknown, not applicable, or invalid.
- Provide administrator pages for maintaining catalogue records and rule definitions.
- Keep an assessment snapshot so its profile values and rule version can be understood later.

1.4 Scope

The system supports information discovery and preliminary matching. It does not submit applications, verify a person's documents, or decide whether a government authority will grant a benefit. Users must check current requirements with the official authority. The supplied seed data is fictional demonstration content.

2. Requirements and scope

2.1 User roles

Role	Main actions
Visitor	Search and filter active schemes; read scheme details, sources, and review status.
Registered user	Update an optional profile, run assessments, read explanations, bookmark schemes, view personal history, and manage account settings.
Administrator	Maintain schemes, categories, eligibility rules, documents, sources, publication status, and audit history.

2.2 Functional requirements

The public catalogue supports keyword search and category filtering. Scheme records are loaded from MySQL rather than embedded in page templates. A signed-in user can save profile details and request an assessment. The result page shows condition-level outcomes and identifies missing information. The administrator area provides forms for maintaining the data used by the catalogue and rule engine.

2.3 Non-functional requirements

- Run locally on a Windows computer using XAMPP, PHP, and MySQL or MariaDB.
- Use server-side validation and prepared database queries.
- Protect user and administrator actions with authentication, authorization, and CSRF checks.
- Keep the interface usable on common desktop and mobile browser sizes.
- Make uncertainty visible instead of assuming a missing profile value is false or eligible.

2.4 Scope decisions

The project does not include email delivery, password recovery, scheme CSV import, or an official data verification service. Those features require additional services or review workflows. The report describes only functions present in the current application.

3. System design

3.1 Application structure

The browser sends requests to a PHP front controller. PHP checks the session and route, validates submitted values, reads or updates records through PDO, and renders the response. The eligibility service evaluates rule records separately from the page templates. MySQL or MariaDB stores accounts, profiles, schemes, categories, rules, documents, sources, bookmarks, assessments, and audit entries.

Part	Responsibility
Browser	Displays responsive pages and submits forms.
PHP application	Routes requests, validates input, applies access checks, and coordinates data changes.
Eligibility engine	Checks typed rule definitions against profile values and produces explanations.
MySQL or MariaDB	Stores application records and saved assessment results.

3.2 Main data relationships

A user has a profile and may have multiple bookmarks and assessments. A scheme can be associated with categories and can have multiple rule groups, rules, documents, and source records. An assessment belongs to a user and scheme and stores the result for each evaluated condition. Administrative actions can create audit entries.

The initial schema is supplied as a versioned SQL migration. A separate seed file can add fictional demonstration records. Keeping the seed separate allows the database to be created without implying that example schemes are verified.

3.3 Assessment sequence

Step	What the application does
1. Select	The user opens an active scheme and requests a preliminary check.
2. Load	The application reads the saved profile and the scheme's active rule definition.
3. Evaluate	Typed operators evaluate each condition within its AND or OR group.
4. Explain	The result page shows each condition outcome and saves a profile snapshot and rule version.

4. Implementation

4.1 Technology

Technology	Use in the project
PHP 8.2	Server-side pages, session authentication, validation, and application logic.
PDO	Database access with prepared statements.
MySQL / MariaDB	Relational storage for application and assessment data.
HTML, CSS, JavaScript	Page structure, responsive styling, and small browser helpers.
PHP unit runner	Zero-dependency checks for the eligibility engine.

4.2 Eligibility rule engine

Each rule identifies a profile attribute, a supported operator, an expected value, an explanation, and its logical group. The engine checks the rule type before comparing values. It does not evaluate PHP code or SQL expressions stored in the database.

Outcome	Meaning
PASS	The available profile value satisfies the configured condition.
FAIL	The available profile value does not satisfy the configured condition.
UNKNOWN	A required value is missing or cannot be evaluated.
NOT APPLICABLE	The rule structure says this condition does not apply.
INVALID	The rule definition has an unsupported or malformed value.

Supported comparisons include equality, numeric boundaries, inclusive ranges, list membership, boolean checks, existence checks, and date comparisons. AND and OR groups combine individual results. Optional rules and conditions that do not apply are handled separately from mandatory failures.

4.3 Security and privacy

- Passwords are stored using PHP password hashing and checked with password verification.
- State-changing forms check CSRF tokens; database queries use prepared statements.
- Protected routes check the signed-in user and required role on the server.
- Rendered values are escaped, and user profile, bookmark, and assessment queries are scoped to the signed-in account.
- Profile fields are optional where practical. The application does not require document uploads for a preliminary check.

These controls are part of an educational prototype. They are not a security audit or legal compliance certification.

5. Website pages and demonstration

The following pages are from the running local application. The demonstration database contains fictional records and displays a DEMO ONLY notice. These images are intended to show the interface and workflow, not to validate government scheme information.

[Website screenshots will be inserted here after the screenshot files are supplied.]

5.1 Demonstration workflow

The local demonstration used an isolated database with fictional scheme records. The public catalogue and scheme detail pages loaded successfully. An administrator signed in and opened the dashboard. A demo user profile was saved, and the application generated a preliminary result for one fictional scheme. The result showed condition-level outcomes and retained the DEMO ONLY notice.

The demonstration profile and result were created only in the temporary local database. They are not evidence of a real person’s eligibility or a real scheme decision.

6. Testing and results

Check	Result
PHP syntax validation	Passed for application and test PHP files using PHP 8.2.
Eligibility engine unit cases	18 cases passed with the project's zero-dependency PHP runner.
Rule cases	Numeric boundaries, missing values, AND/OR groups, ranges, optional conditions, applicability, unsupported operators, and malformed groups.
Local browser workflow	Home, catalogue, scheme details, admin dashboard, profile save, and one assessment were exercised against an isolated demo database.

The browser workflow confirms those pages and actions worked in the local demonstration. It does not prove performance under load, current accuracy of public scheme information, or production security. The repository's manual test plan contains additional scenarios for account permissions, search, bookmarks, history, administrator edits, and invalid input.

7. Installation and limitations

7.1 Local setup

1. Copy the project folder into C:\xampp\htdocs\GovernmentSchemesMatcher.
2. Start Apache and MySQL in the XAMPP Control Panel.
3. Import database/migrations/001_initial_schema.sql in phpMyAdmin.
4. Optionally import database/seeds/001_demo.sql for fictional demonstration records.
5. Copy config/database.example.php to config/database.local.php and enter the local database settings.
6. Create an administrator with scripts/create_admin.php; the script prompts for a password.
7. Open http://localhost/GovernmentSchemesMatcher/public/.

7.2 Current limitations

- The supplied scheme examples are fictional and must not be treated as current public-service information.
- There is no email delivery or password recovery flow.
- The project has no CSV import or automated browser/database integration suite.
- A complete manual test run in the student's XAMPP environment remains advisable before submission or demonstration.
- The system gives preliminary comparisons only and does not replace an official application or decision.

Conclusion

The project implements a PHP and MySQL application for maintaining scheme records and comparing configured rules with saved profile details. Its main feature is an explainable result for each rule, including a separate state for missing information. Unit checks and a local browser demonstration confirmed the rule engine and the main visible workflow. Before using the system with real scheme data, each record and source would need to be reviewed and kept current.

References

- Government Schemes Matcher and Eligibility Tester project brief supplied for this project.
- Project README and documentation: docs/SRS.md, docs/ARCHITECTURE.md, docs/DATABASE.md, docs/INSTALLATION.md, docs/TEST_PLAN.md, docs/USER_GUIDE.md, and docs/ADMIN_GUIDE.md.
- PHP Manual. https://www.php.net/manual/
- MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/